

Mini Project: Build a RAG System for Your Own Data

Practice on the course pipeline, then create your own documents and questions and build a RAG for them.

Runs on your own machine or Colab • 100 points

1. What this project is about

In the lectures you saw a RAG system built in two stages. The first notebook covered retrieval: chunking, embeddings, and the vector database. The second assembled the full pipeline, from a question to a grounded answer. Here you do the same thing on data of your own.

Part 1 is practice: five short pieces of code that rebuild the lecture pipeline on the course documents. The rest is the project proper, and it is where the credit is. You create your own documents, write your own questions, build a RAG for them with no code provided, run experiments, and explain what you found in a report and a video.

Everything graded runs offline on a small local embedding model, with no API key and no GPU. If you have a key or a local model, the generation step will use it.

2. Using AI assistants

You may use ChatGPT, Claude, Copilot, or any other assistant to write code, fix errors, and organize results. What is assessed is whether you understand the system you built: in the video you will walk through your own notebook and explain what each part does and why you chose the settings you chose. Anything you cannot explain will not earn credit, however well it runs.

3. Part 1: practice on the course pipeline

10 points

Open `MiniProject_RAG_Starter.ipynb`. Parts 1 to 5 contain five short pieces of code, each marked with a `TODO` banner. Everything you need is in the lecture notebooks.

	You write	Lines	From
TODO 1	cut a document into overlapping chunks	3	Lecture notebook 1
TODO 2	embed the chunks and store them in ChromaDB	2	Lecture notebook 1
TODO 3	retrieve the closest chunks, dropping weak matches	3	Lecture notebook 2
TODO 4	build the context block for the prompt	2	Lecture notebook 2
TODO 5	count how many answers were found and refusals kept	4	Lecture notebook 2

Each cell states the output to expect: 22 chunks after TODO 1, and found 6/6 with refused 2/2 after TODO 5. Fix any number that does not match before moving on, because a retrieval bug does not crash, it quietly gives you wrong numbers.

4. Part 2: create your data and your questions

15 points

Choose a topic you know something about, in cybersecurity or elsewhere, where you can write questions with checkable answers. Your documents go in a folder `my_kb/`: 4 to 6 files (`.txt` or `.md`), roughly 10,000 characters in total. Write them yourself, as notes or procedures, or collect public documents such as OWASP cheat sheets or CISA advisories. Record where each came from, or that you wrote it, in `my_kb/SOURCES.md`.

Your questions go in `goldset.json`, using `goldset_template.json` as the pattern. Write eight:

- Six that your documents answer. For each, copy a short phrase from the document exactly as written and use it as the `evidence`. That is how you check whether retrieval found the right text. Keep it under about 80 characters and on one line, or a chunk boundary may cut it in half and report a miss that is not real.
- Two that your documents do not answer: one clearly unrelated, and one that sounds like your topic but is genuinely not in your files. The second is the real test. A system that answers it anyway is making things up.

5. Part 3: build a RAG for your own data

25 points

From Part 6 of the notebook onward, no code is provided. Reuse what you wrote in Part 1, adapt the lecture notebooks, or write something different; what matters is that it works on your data and that you can explain it.

Your notebook must load your documents and questions, check that your evidence phrases really appear in your documents, chunk and embed the documents into a vector database, retrieve relevant chunks, build a prompt from them, and produce grounded answers. Print a transcript of your system answering several of your questions, including at least one it should refuse.

6. Part 4: experiment and collect data

15 points

Change one setting at a time, such as chunk size, the number of chunks retrieved, or the distance threshold, and score each configuration against your eight questions, recording answers found and refusals held. Collect the results in a table, save it as `results.csv`, and draw at least one chart. Watch the refusals as closely as the answers: a setting that finds more answers but refuses less is not an improvement, it is a system that has started guessing.

7. What to hand in

Submit this

one PDF report. On its first page, put the link to your project folder on GitHub and the link to your video. Everything else lives in the repository.

7.1 Your repository (5 points)

Create a private GitHub repository and add your instructor and the class TA as collaborators. Do not give access to other students. Put this project in a folder of its own, structured like this:

```
your-repo/
├── rag_project/
│   ├── README.md           what the project is, in a short paragraph
│   ├── rag_notebook.ipynb  your completed notebook, with all outputs saved
│   ├── goldset.json        your eight questions
│   ├── results.csv         your experiment results
│   ├── figure.png         your chart
│   └── my_kb/
│       ├── SOURCES.md     where each document came from
│       ├── doc1.md
│       └── doc2.md        your documents
```

Run the notebook top to bottom and save it with its outputs showing before you commit. GitHub displays saved outputs, so a reader sees your results without running anything, and a notebook committed with empty cells shows nothing.

After grading you are encouraged to make the repository public. One aim of this course is that you finish with a portfolio of AI projects an employer can look at, so build this one to be readable by a stranger.

7.2 Report (15 points)

A PDF containing:

1. How your RAG system works, in your own words and end to end: what an embedding is and why it allows searching by meaning, why documents are chunked and what goes wrong when they are too large or too small, what the vector database stores and returns, and the three steps of RAG, pointing to where each appears in your notebook.
2. What you built: your topic, your documents, your eight questions, and why you chose the question your documents cannot answer.
3. What your experiments showed: your table and chart, which setting mattered most, and the settings you would use.
4. One question your system got wrong. Look at the chunk it retrieved instead and explain the mistake.
5. What your evidence does not tell you. Six answerable questions means one question is a sixth of your score.

7.3 Video (15 points)

Five to ten minutes of screen recording with your narration. Walk through your notebook while it runs, explain the pipeline in your own words, show your system answering one question and refusing another, and talk through your results. Live screens, not slides of screenshots.

8. Grading

Part	Points
Part 1, practice pipeline: five TODOs, 22 chunks, found 6/6, refused 2/2	10
Part 2, your documents and your eight questions	15
Part 3, your own RAG running end to end on your data	25
Part 4, experiments, <code>results.csv</code> , and a chart	15
Repository: private, instructor and TA added, clear structure, outputs saved	5
Report	15
Video	15
Total	100

Use only documents you may share, or text you wrote yourself, and cite them in `SOURCES.md`. Keep API keys in `.env` or Colab Secrets, never in the notebook or the repository. Build the pipeline yourself; LangChain is the next module.